

Introduction to Jenkins

Module 5: Shared Libraries



Topics

- Why Shared Libraries matter
- Library structure and best practices
- Global variables and helper utilities
- Versioning and dependency strategy



Shared Libraries

- Without shared libraries As Jenkins adoption grows
 - Pipelines become long and repetitive
 - Teams copy and paste logic
 - Fixes must be applied everywhere
 - Standards drift over time
 - Security practices become inconsistent
- This leads to
 - Maintenance pain
 - Increased risk
 - Hard-to-govern pipelines



Shared Libraries

- Shared libraries support
 - Centralize reusable pipeline logic
 - Enforce best practices
 - Reduce duplication
 - Improve security and maintainability
- Logic is encapsulated in library calls
 - Explicit execution logic
 - *sh 'mvn clean verify'*
 - Replaced by a function call
 - *RunMavenBuild()*
 - Pipelines describe intent; shared libraries implement behaviors



Library Structure

- A Jenkins shared library is a Git repository
 - It has a standard well-defined layout
 - Not all directories are required
 - This structure is a best practice.
- `vars/`
 - Global Pipeline Steps
 - Contains shared steps exposed directly to pipelines
 - One file = one callable step
 - File name becomes the step name
 - Best practice
 - *Keep logic here simple*
 - *Delegate complexity to `src/`*

```
(shared-library-repo)
├─ vars/
├─ src/
├─ resources/
└─ README.md
```

```
runTests()
deployToEnv('prod')
```



Library Structure

- src/
 - Helper Utilities and Classes
 - Contains Groovy classes
 - Used internally by shared steps
 - Not exposed directly to Jenkinsfiles
- Use this for
 - Validation logic
 - Parsing
 - Reusable utilities
 - Complex workflows
- Analogy
 - vars/ is the API
 - src/ is the implementation

```
shared-library/  
├─ vars/  
│   └─ runMavenBuild.groovy  
└─ src/  
    └─ com/example/build/  
        └─ MavenHelper.groovy
```

```
package com.example.build  
  
class MavenHelper {  
  
    static void runBuild(script, String goals) {  
        script.echo "Running Maven goals: ${goals}"  
        script.sh "mvn ${goals}"  
    }  
  
    static void validateGoals(String goals) {  
        if (!goals) {  
            throw new IllegalArgumentException("Maven goals must not be empty")  
        }  
    }  
}
```



Library Structure

- resources/
 - Templates and Assets
 - Stores non-code assets
 - *YAML, JSON, XML, shell scripts, templates*
 - Loaded at runtime
- Common uses
 - Kubernetes manifests
 - Deployment templates
 - Configuration files

```
shared-library/  
├─ resources/  
│   ├─ docker/  
│   │   └─ Dockerfile.template  
│   ├─ kubernetes/  
│   │   ├─ deployment.yaml  
│   │   └─ service.yaml  
│   └─ scripts/  
│       └─ deploy.sh  
└─ config/  
    └─ sonar-project.properties  
└─ README-template.md
```



Library Structure

- How resources are loaded
 - The libraryResource step loads files from resources/
 - In the example, the configuration files needed are created from the corresponding resource
 - Avoids hard coding files or strings
- Best practices
 - Don't put complex logic directly in Jenkinsfiles
 - Don't expose too many global steps
 - Don't duplicate logic across libraries
 - Don't let every team modify shared libraries

```
def deploymentYaml = libraryResource('kubernetes/deployment.yaml')

writeFile file: 'deployment.yaml', text: deploymentYaml
sh 'kubectl apply -f deployment.yaml'
```

```
def script = libraryResource('scripts/deploy.sh')

writeFile file: 'deploy.sh', text: script
sh 'chmod +x deploy.sh && ./deploy.sh'
```



Versioning Strategies

- Without versioning
 - Library changes can break pipelines
 - Debugging becomes difficult
 - Rollbacks are painful
- Shared libraries should be
 - Versioned
 - Predictable
 - Reproducible



Versioning Strategy

- Common approaches
 - Git tags (v1.0.0)
 - Branches (main, stable)
 - Semantic versioning
- Pipelines can
 - Lock to a version
 - Upgrade intentionally
 - Test changes safely



Managing Library Dependencies

- Best practices
 - Minimize library dependencies
 - Avoid circular dependencies
 - Keep libraries focused
 - Prefer one “core” shared library
 - Shared libraries should evolve slowly and carefully
- Governance and control in enterprise setups:
 - Library changes require review
 - CI tests validate libraries
 - Releases are documented
 - Breaking changes are communicated
 - Shared libraries become part of the platform.



Questions

